FREE GUIDE

Source Code Analysis / April 2026

27 Things Anthropic Does With Claude Code That You Don't

When Claude Code's source was briefly exposed via npm, the community surfaced how Anthropic engineers configure their own tool. Based on what was publicly shared that day, here are 27 configurations you can implement today.

27	6	20+	4
CONFIGS	SECTIONS	HOOK EVENTS	MEMORY LEVELS

GTM Engineer
gtmengineer.be

Disclaimer: This guide is for educational purposes only. We do not possess, host, or distribute any leaked source code. All insights are based on publicly available information shared on social media and open forums on March 31, 2026. All configurations described are available in the current public release of Claude Code.

SECTION 01

Memory That Actually Works

Most Claude Code users treat every session as a blank slate. Anthropic doesn't.

1 The 4-Level Memory Hierarchy

Claude Code loads instructions from four levels, each overriding the last:

PRIORITY	FILE	SCOPE

4 (highest)	.claude/CLAUDE.local.md	Local, private
3	CLAUDE.md + .claude/CLAUDE.md + .claude/rules/*.md	Project
2	~/.claude/CLAUDE.md	Personal, all projects
1 (lowest)	/etc/claude-code/CLAUDE.md	Global

> **WHAT TO DO:** Create all four. Personal preferences in ~/.claude/CLAUDE.md. Project conventions in the project-level file. Sensitive context in .claude/CLAUDE.local.md.

2 The @include Directive

Reference external files from any CLAUDE.md with @./path, @~/path, or @/path syntax. Supports .md, .txt, .json, .py, .ts, .yaml, and dozens of other text formats.

```
@./docs/coding-standards.md
@./docs/api-conventions.md
@~/shared-rules/security-policy.md
```

> **WHAT TO DO:** Break your CLAUDE.md into focused files and compose them with @includes. One file per concern.

3 File-Scoped Rules with .claude/rules/

Drop individual .md files into .claude/rules/ and they're loaded automatically. No registration needed. One file per concern: testing, migrations, API conventions, security.

> **WHAT TO DO:** Start splitting your monolithic CLAUDE.md into focused rule files.

4 Auto-Memory Extraction

A background system watches your conversations and saves four types of memories: **User** (role, expertise), **Feedback** (corrections), **Project** (deadlines, goals), and **Reference** (external system pointers). Each gets its own file with structured frontmatter.

> **WHAT TO DO:** Enable auto-memory. When Claude gets something right or wrong, tell it to remember. Check ~/.claude/projects/<your-project>/memory/ to see what it saved.

5 Auto-Dream Consolidation

Every 24 hours (after 5+ sessions), a background process consolidates memories in 4 phases: **Orient** (read existing files), **Gather Signal** (scan transcripts), **Consolidate** (merge and deduplicate), **Prune** (keep index under 200 lines).

> **WHAT TO DO:** Enable autoDreamEnabled: true in settings. Let it run for a week, then check your memory directory.

6 The Session Memory Template

When Claude compacts a long conversation, it follows a 10-section template (max 2,000 chars each, 12,000 tokens total). "Current State" is the most critical section for continuity.

1. Session Title
2. Current State (most critical)
3. Task Specification
4. Files and Functions
5. Workflow
6. Errors & Corrections
7. Codebase Documentation
8. Learnings
9. Key Results
10. Worklog

> **WHAT TO DO:** Front-load context about your current state. The clearer your initial context, the better it survives compression.

SECTION 02

Hooks That Automate Everything

The most underutilized feature. Four hook types, 20+ event triggers, and patterns that eliminate manual work.

7 The Four Hook Types

Command — run shell commands. **Prompt** — ask an LLM to evaluate something. **HTTP** — call external APIs/webhooks. **Agent** — spawn a verification agent.

```
// Command hook
{ "type": "command", "command": "npm test --bail", "timeout": 30 }

// Prompt hook
{ "type": "prompt", "prompt": "Is this bash command safe: $ARGUMENTS", "model": "claude-haiku-4-5-20251
001" }

// HTTP hook
{ "type": "http", "url": "https://api.example.com/webhook", "headers": {"Authorization": "Bearer $TOKE
N"} }

// Agent hook
```

```
{ "type": "agent", "prompt": "Verify tests pass with no regressions", "model": "claude-haiku-4-5-202510
01" }
```

WHAT TO DO: Start with command hooks for testing. Graduate to prompt hooks for code review gates. Use HTTP hooks for Slack/CI integration.

8 20+ Event Triggers

EVENT	FIRES WHEN	USE CASE
preToolUse	Before any tool runs	Gate dangerous commands
postToolUse	After tool completes	Auto-test, auto-lint
sessionStart	Session begins	Load context, check env
sessionEnd	Session ends	Cleanup, save state
fileChanged	File is modified	Trigger tests/linting
preCompact	Before compression	Save critical state
userPromptSubmit	User sends message	Input validation
permissionDenied	Permission denied	Alert, log, adapt

WHAT TO DO: Set up postToolUse with a Write matcher to auto-run tests. Use sessionStart to verify your environment.

9 Conditional Hooks with Matchers

Hooks fire only for specific tools. Match "Write" for lint, "Bash" with an "if" condition for tests after commits.

```
{ "hooks": { "postToolUse": [
  { "matcher": "Write", "hooks": [{ "type": "command", "command": "npm run lint --fix" }] },
  { "matcher": "Bash", "hooks": [{ "type": "command", "command": "npm test", "if": "Bash(git commit*)"
}] }
]}}
```

WHAT TO DO: Create targeted hooks: lint after writes, test after commits, notify after deployments.

10 Async Hooks That Don't Block

Add "async": true to run in background. Add "asyncRewake": true to wake Claude when done (exit code 2 triggers response).

```
{ "type": "command", "command": "npm test --silent", "async": true, "asyncRewake": true }
```

WHAT TO DO: Use async for slow operations (full test suites, deployments) so Claude keeps working.

11 The One-Shot Hook Pattern

Add "once": true to fire only once per session. Perfect for environment setup.

```
{ "type": "command", "command": "docker compose up -d", "once": true }
```

WHAT TO DO: Use one-shot hooks to start databases, pull configs, check dependencies at session start.

SECTION 03

Permissions That Eliminate Friction

The "approve every command" experience is the #1 complaint. Here's how to fix it.

12 Pattern-Based Permission Rules

Glob-style matching: * for arguments, ** for paths. Allow safe commands, deny dangerous ones.

```
{ "permissions": { "allow": [
  "Bash(git *)", "Bash(npm test*)", "Bash(npm run lint*)",
  "Bash(ls *)", "Bash(grep *)", "Read(**)", "Edit(**)"
]}}
```

WHAT TO DO: Start by allowing read-only commands and git operations. Gradually expand.

13 The Five Permission Modes

MODE	BEHAVIOR	BEST FOR
default	Ask for every sensitive op	Beginners
plan	Show plan first, then ask	Code review
acceptEdits	Auto-approve file edits	Daily dev work
dontAsk	Auto-approve everything	CI pipelines

bypassPermissions	Skip all checks	Testing only

> **WHAT TO DO:** Use acceptEdits daily. Switch to plan for complex refactors. Use dontAsk in CI.

14 Project-Level Permission Overrides

Put rules in .claude/settings.json at project root. Commits to version control so every team member gets the same defaults.

> **WHAT TO DO:** Create a shared .claude/settings.json with allow/deny rules for your team.

SECTION 04

Environment Variables That Unlock Hidden Features

15 Effort Levels

Control how hard Claude thinks. Low for quick lookups, medium (default) for daily work, high for complex debugging.

```
export CLAUDE_CODE_EFFORT_LEVEL=high  # Options: low, medium, high
```

> **WHAT TO DO:** Use low for quick questions. Use high for architecture decisions and multi-file refactors.

16 Context Window Control

```
export CLAUDE_CODE_MAX_CONTEXT_TOKENS=800000
export CLAUDE_CODE_MAX_OUTPUT_TOKENS=16000
```

> **WHAT TO DO:** Increase context if compacting too often. Increase output if responses are cut off.

17 Subagent Model Override

Use Opus for main conversation, Haiku for research subagents. Saves money without quality loss.

```
export CLAUDE_CODE_SUBAGENT_MODEL=claude-haiku-4-5-20251001
```

18 Debug Logging

```
export CLAUDE_CODE_DEBUG_LOGS_DIR=~/.claude/debug-logs
```

> **WHAT TO DO:** Enable when troubleshooting. See permission decisions, compaction triggers, memory extraction.

19 Disable Features You Don't Need

```
export CLAUDE_CODE_DISABLE_AUTO_MEMORY=1
export CLAUDE_CODE_DISABLE_CLAUDE_MDS=1
export DISABLE_AUTO_COMPACT=1
export CLAUDE_CODE_DISABLE_BACKGROUND_TASKS=1
```

> **WHAT TO DO:** Disable unused features for faster startup and less background CPU.

20 The Bare Mode Flag

Skips hooks, LSP, plugins, keychain. Pure, fast, minimal Claude Code.

```
claude --bare
# or: export CLAUDE_CODE_SIMPLE=1
```

SECTION 05

Agent Orchestration

21 Custom Agents in .claude/agents/

Create reusable .md files defining agent personality, tools, and constraints. Security reviewers, test writers, docs generators — defined once, spawned by name.

22 Worktree Isolation

Agents work in isolated git worktrees. Symlink heavy dirs (node_modules), use sparse paths for monorepos.

```
{ "worktree": { "symlinkDirectories": ["node_modules", ".venv"], "sparsePaths": ["src/", "tests/"] }}
```

23 Coordinator Mode

One Claude plans, multiple workers execute in parallel with constrained tool access.

```
export CLAUDE_CODE_COORDINATOR_MODE=1
```

> **WHAT TO DO:** Try coordinator mode for large refactoring tasks.

SECTION 06

UI & Workflow Optimization

24 Custom Status Line

```
{ "statusLine": { "type": "command", "command": "echo $(git branch --show-current) $(git status --porce
lain | wc -l) files changed" }}
```

25 Custom Spinner Verbs

```
{ "spinnerVerbs": { "mode": "append", "verbs": ["Deploying", "Refactoring", "Debugging"] }}
```

26 Attribution Control

Remove or customize Co-Authored-By lines and PR badges. Set to empty strings to hide completely.

```
{ "attribution": { "commit": "", "pr": "" }}  // hide all
{ "attribution": { "commit": "Assisted by AI", "pr": "Partially AI-generated" }}  // custom
```

27 The Complete Power User Settings

```
{
  "model": "claude-opus-4-6",
  "effortLevel": "medium",
  "fastMode": true,
  "alwaysThinkingEnabled": true,
  "autoDreamEnabled": true,
  "showTurnDuration": true,
  "permissions": {
    "defaultMode": "acceptEdits",
    "allow": ["Bash(git *)", "Bash(npm test*)", "Bash(npm run lint*)", "Read(**)", "Edit(**)"],
```

```
    "deny": ["Bash(rm -rf *)", "Bash(*sudo*)", "Bash(git push --force*)"]
  },
  "hooks": {
    "postToolUse": [{ "matcher": "Write", "hooks": [{ "type": "command", "command": "npm run lint --fix
--quiet", "async": true }] }],
    "sessionStart": [{ "matcher": null, "hooks": [{ "type": "command", "command": "echo Branch: $(git b
ranch --show-current)", "once": true }] }]
  },
  "worktree": { "symlinkDirectories": ["node_modules", ".venv"] },
  "statusLine": { "type": "command", "command": "echo $(git branch --show-current)" }
}
```

WHAT TO DO: Copy to ~/.claude/settings.json. Adjust allow/deny for your stack. Add hooks for your test runner. Iterate weekly.

The Bottom Line

Anthropic's engineers don't use Claude Code out of the box. They configure 4-level memory hierarchies. They set up hooks that auto-test, auto-lint, and auto-notify. They fine-tune permissions so Claude can move fast without asking for approval on every git status.

The leaked source didn't just reveal how Claude Code works — it revealed how much of its power is hidden behind configuration that most users never touch.

Now you know. Go configure it.

Want More Claude Code for GTM Tips?

I share weekly insights on using Claude Code for go-to-market engineering — automation hooks, agent workflows, memory patterns, and real configurations that save hours.

Connect on LinkedIn →

This guide is for educational purposes only. Based on publicly shared information from March 31, 2026. No leaked source code is possessed or distributed.

Website	LinkedIn	GitHub
gtmengineer.be	linkedin.com/in/gtm-architect	@promptgtm